Distributed MapReduce on Lab Machines

SLR Project P4

Project presentation

10-minute discussion

One-sentence summary

A custom Go MapReduce framework running on `*.enst.fr` workers, with direct Common Crawl input, pluggable jobs, local-disk shuffle, and fault-aware coordination.

What We Built

- Automatic host discovery from `https://tp.telecom-paris.fr/ajax.php`
- Go orchestrator that builds, deploys, starts, monitors, and cleans workers
- HTTP worker API for `load -> map -> shuffle -> reduce -> result`
- Direct Common Crawl WET download from `data.commoncrawl.org`
- Four jobs: `wordcount`, `langdetect`, `domainpop`, `docdensity`

Current verification

89 Go tests passed, live 100-host discovery passed, and a small reference wordcount matched exactly.

Architecture

```
User shell
  |
  v
mapreduce.sh
  |-- make_hosts -> hosts.txt
  |
  v
Go orchestrator
  |-- build worker with selected job
  |-- SCP + SSH start workers
  |-- health checks and retries
  |-- load / map / reduce / collect
  v
Remote workers on lab machines
  /health /data /load /map /intermediate /reduce /result
```

Design choice: the client is the Main. Workers are stateless between jobs except for local temporary files under `/tmp`.

Protocol and Dataflow

1. **Load**: local file chunks via `/data`, or Common Crawl WET URLs via `/load`
2. **Map**: each worker writes `N` bucket files on local disk
3. **Shuffle + reduce**: reducer `i` pulls bucket `i` from every peer via `/intermediate`
4. **Collect**: Main fetches `/result`, merges integer values, sorts, writes final output

Reducer assignment

```
FNV-32a(key) % N
```

Fault Tolerance and Operations

- Logical **slots** stay fixed: `0..N-1`
- Physical hosts can be replaced by cold spares
- Health watcher polls `/health` and marks repeated failures dead
- Failed slots replay prerequisites: `load -> map -> reduce`
- If a peer dies during reduce, coordinator re-runs stale reduces with the new peer list
- Cleanup is idempotent: kill PID if present, remove worker files, tolerate repeated cleanup

Tested behavior

Coordinator replacement, peer reduce failure, cold spare activation, and fallback to a surviving worker.

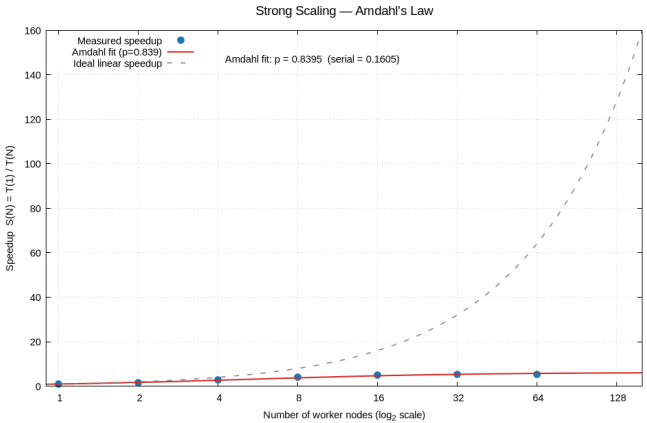
Use Cases Beyond Wordcount

Job	What it measures	Example output
`wordcount`	Token frequency	`the 239298`
`langdetect`	Language signal counts	`lang:en`, `lang:fr`
`domainpop`	Crawled domain popularity	`doi.org`, `youtube.com`
`docdensity`	Content size and density	`stat:lines`, `hist:len.*`

Framework property

All reducers emit integer values, so the orchestrator can safely merge partial outputs across jobs.

Measurements and Amdahl's Law



- 1 node: 206.95 s, speedup 1.00
- 16 nodes: 40.80 s, speedup 5.07
- 64 nodes: 39.06 s, speedup 5.30

Interpretation

After useful parallelism is exhausted, communication, synchronization, and final merge dominate.

Batch MapReduce vs Kafka Streams

Custom MapReduce	Kafka Streams
Batch job with explicit phases	Continuous stream processing model
Lightweight HTTP + local temp files	Persistent Kafka log and state stores
Fast on small bounded workloads	More operationally robust at scale
No HDFS-style replication	Built around durable broker storage

Kafka comparison path

Kafka runs without Docker or root: KRaft deployment in `/tmp`, Java Streams wordcount, timing by consumer-group lag.

What Is Missing and One-Week Plan

Gap	One-week fix
Exact `1 SCP to HOME + 100 SSH` rubric mode	Add compatibility deploy mode or document design choice
Port collision preflight	Remote port check before starting workers
Standalone MapReduce cleanup script	Add `clean_mapreduce.sh` using host file
`/cal/commoncrawl` evidence	Add input mode or document direct Amazon path as replacement
Local storage proof	Save `df` / `mount` evidence for lab machines
Large remote proof	Run and save 100-node / larger-split logs
Stragglers and atomic final writes	Add speculative task note or implement temp+rename outputs

Presentation stance

The core system works; remaining work is mostly operational hardening and evidence collection.

Demo and Discussion Plan

Live demo path

```
(cd scripts && go test ./...)
./mapreduce.sh -job scripts/jobs/wordcount \
  -input test_input.txt -n 4 -output result.txt
./run_all_commoncrawl_jobs.sh -crawl CC-MAIN-2026-21 \
  -files-limit 1 -n 4
```

Backup if machines are unstable

- Passed tests and reference validation
- Checked-in outputs and timing CSVs
- Amdahl plot and bottleneck explanation

Suggested speaking split

- Speaker 1: architecture and protocol
- Speaker 2: jobs, results, Kafka
- Speaker 3: fault tolerance, roadmap, Q&A